Hibernate Internals

Hibernate becomes much easier to understand once we stop seeing it as a tool that only generates SQL. Internally, Hibernate creates and manages a working area in memory, tracks entity objects inside that area, detects changes, and synchronizes those changes with the database at the correct time.

This chapter covers:

- `EntityManagerFactory` and `EntityManager`
- Persistence Context
- `@PersistenceContext`
- Transactions and `@Transactional`
- Entity lifecycle states
- Dirty checking
- Flushing and write-behind
- First-level cache

---

1. The Core Hibernate Components

Component	Responsibility	Typical Lifetime
`EntityManagerFactory`	Creates `EntityManager` instances and stores shared ORM metadata	Entire application
`EntityManager`	Provides operations for interacting with entities and the persistence context	One unit of work or transaction
Persistence Context	Stores and tracks managed entity instances	Usually one transaction
Database Transaction	Ensures a group of database operations succeeds or fails as one unit	One business operation

The basic relationship is:

```
EntityManagerFactory
        |
        | creates
        ▼
EntityManager
        |
        | interacts with
        ▼
Persistence Context
        |
        | synchronizes through Hibernate
        ▼
Database
```

The factory is expensive to create and is normally shared for the lifetime of the application. An actual `EntityManager` is short-lived and must not be shared concurrently between threads.

---

2. JPA, Hibernate, and Spring Boot

JPA: The Specification

Jakarta Persistence, commonly called JPA, defines a standard contract for object-relational mapping in Java. It provides interfaces, annotations, and rules such as:

- `EntityManagerFactory`
- `EntityManager`
- `@Entity`
- `@Id`
- `@GeneratedValue`

JPA defines what persistence operations should do. It does not contain the complete ORM engine that performs those operations.

Hibernate: The Provider

Hibernate is a JPA provider. It implements the JPA contract and performs the actual work:

- mapping entity objects to tables,

- generating SQL,
- interacting with JDBC,
- tracking entity state,
- performing dirty checking,
- managing the persistence context,
- coordinating synchronization with the database.

Hibernate also offers its own native API:

JPA API	Hibernate Native API
`EntityManagerFactory`	`SessionFactory`
`EntityManager`	`Session`

In Hibernate, `SessionFactory` extends `EntityManagerFactory`, and `Session` extends `EntityManager`. This allows Hibernate to support the standard JPA API while also providing Hibernate-specific features.

Spring Boot's Role

In a Spring Boot application, our code normally uses JPA types while Hibernate works underneath as the provider.

```
import jakarta.persistence.Entity;
import jakarta.persistence.EntityManager;
import jakarta.persistence.Id;
```

Programming against JPA keeps most persistence code provider-independent. In principle, Hibernate could be replaced by another JPA provider with fewer changes than if the application directly depended on Hibernate-specific APIs.

> Spring Boot configures the persistence infrastructure, JPA provides the standard API, and Hibernate implements the ORM behavior.

---

3. The Persistence Context

A persistence context is Hibernate's in-memory workspace for managed entities.

```
Persistence Context
┌──────────────────────────────────┐
│ Student#1 → Student object       │
│ Student#2 → Student object       │
│ Student#7 → Student object       │
│                                  │
│ Tracks identity and state        │
└──────────────────────────────────┘
```

Suppose we execute:

```java
Student student =
        entityManager.find(Student.class, 1L);
```

If the entity is not already available in the current persistence context, Hibernate may:

1. execute a `SELECT`,
2. create a `Student` object from the returned row,
3. place that object in the persistence context,
4. begin managing its state.

Conceptually:

```
(Student.class, 1L)
          |
          ▼
Student {
    id = 1,
    name = "Rahul",
    email = "rahul@gmail.com"
}
```

The persistence context is responsible for much more than storing objects. It supports:

- entity identity,

- lifecycle state management,
- automatic change detection,
- first-level caching,
- transactional write-behind,
- synchronization with the database.

The Real Purpose of `EntityManager`

`EntityManager` is not merely a class that runs database queries. It is the main JPA interface through which we interact with a persistence context.

```
entityManager.persist(student);
entityManager.find(Student.class, 1L);
entityManager.remove(student);
entityManager.flush();
entityManager.detach(student);
entityManager.clear();
entityManager.refresh(student);
```

Each operation affects an entity, the persistence context, or the synchronization between the persistence context and the database.

---

4. `@PersistenceContext` in Spring

An `EntityManager` can be injected into a Spring repository:

```
@Repository
public class StudentRepository {

    @PersistenceContext
    private EntityManager entityManager;
}
```

The name `@PersistenceContext` is meaningful. We are asking Spring for an `EntityManager` connected to the persistence context that is appropriate for the

current operation.

Why Spring Does Not Inject One Permanent `EntityManager`

Spring repositories are singleton beans by default. The same repository object can therefore be used by many request threads.

An actual JPA `EntityManager`, like a Hibernate `Session`, is not thread-safe. Sharing one physical instance across concurrent requests would be unsafe.

Spring normally injects a shared proxy instead:

```
Singleton Repository
        |
        ▼
Shared EntityManager Proxy
        |
        | routes the call
        ▼
Transaction-associated EntityManager
        |
        ▼
Persistence Context
```

Two concurrent transactions can therefore use the same repository while working with different actual persistence contexts:

```
Request A → Transaction A → EntityManager A → Persistence Context A

Request B → Transaction B → EntityManager B → Persistence Context B
```

The injected proxy is stable, but the actual `EntityManager` handling a call depends on the current transactional context.

---

5. Transactions from First Principles

Consider a bank transfer:

```sql
UPDATE accounts
SET balance = balance - 100
```

```
WHERE id = 1;

UPDATE accounts
SET balance = balance + 100
WHERE id = 2;
```

If the first statement succeeds and the second fails, money is deducted from one account but never added to the other.

A transaction treats both statements as one logical operation:

```
BEGIN
  UPDATE account A
  UPDATE account B
COMMIT
```

If any operation fails:

```
BEGIN
  UPDATE account A
  UPDATE account B  ← error
ROLLBACK
```

The region between beginning and completing the transaction is called the **transaction boundary**.

Unit of Work

A unit of work is the complete set of persistence actions required to perform one business operation.

For example, changing a student's email may involve:

1. retrieving the student,
2. checking business rules,
3. modifying the entity,
4. detecting the change,

5. generating SQL,
6. executing SQL,
7. committing the transaction.

All these steps belong to one logical unit of work.

The transaction defines whether the database changes succeed together. The persistence context tracks the entities participating in that work.

Database, Hibernate, and Spring Transactions

These are different layers around the same transactional work.

Database Transaction

The database understands commands such as:

```
BEGIN;
UPDATE students SET email = 'new@email.com' WHERE id = 1;
COMMIT;
```

Hibernate Transaction

Using Hibernate's native API, transaction management can be written manually:

```
Session session = sessionFactory.openSession();
Transaction transaction = session.beginTransaction();

try {
    session.persist(student);
    transaction.commit();
} catch (Exception exception) {
    transaction.rollback();
    throw exception;
} finally {
```

```
    session.close();
}
```

Spring Transaction

Spring removes the repetitive transaction-management code:

```
@Transactional
public void createStudent(Student student) {
    entityManager.persist(student);
}
```

Conceptually, Spring performs:

```
Begin transaction
        ↓
Execute method
        ↓
Flush pending changes
        ↓
Commit
```

If the method fails with an exception that triggers rollback:

```
Begin transaction
        ↓
Execute method
        ↓
Exception
        ↓
Rollback
```

By default, Spring rolls back for unchecked exceptions such as `RuntimeException` and `Error`. Rollback behavior for checked exceptions can be configured using attributes such as `rollbackFor`.

---

6. How `@Transactional` Works

Spring normally applies `@Transactional` through an AOP proxy.

```
@Service
public class StudentService {

    @Transactional
    public void updateStudent(Long id) {
        // business operation
    }
}
```

Conceptual call flow:

```
Controller
    |
    ▼
StudentService Proxy
    |
    | TransactionInterceptor begins transaction
    ▼
StudentService.updateStudent()
    |
    | method completes or throws an exception
    ▼
Commit or rollback
```

Because the proxy controls the transaction boundary, a call made from one method to another method on the same object may bypass the proxy. This is the usual self-invocation limitation of Spring proxy-based AOP.

---

7. Complete Transaction and Persistence Context Flow

Consider:

```
@Transactional
public void updateStudent(Long id) {

    Student student =
```

```
            entityManager.find(Student.class, id);

    student.setName("Rohan");
}
```

Step 1: The Call Reaches the Service Proxy

```
Controller → StudentService Proxy
```

Step 2: Spring Starts the Transaction

The transaction interceptor creates or joins a transaction according to the propagation rules.

Step 3: An `EntityManager` Is Associated with the Transaction

```
Transaction
    |
    ▼
EntityManager
    |
    ▼
Persistence Context
```

Step 4: Hibernate Executes `find()`

Hibernate first checks the persistence context for `Student#id`.

- If present, it returns the managed instance.
- If absent, it queries the database, creates the entity, and manages it.

Step 5: The Managed Entity Is Modified

```
student.setName("Rohan");
```

No explicit update method is required. Hibernate tracks the managed object.

Step 6: Hibernate Flushes

Before commit, Hibernate normally synchronizes pending persistence-context changes with the database.

```
UPDATE students
SET name = 'Rohan'
WHERE id = ?;
```

Step 7: The Database Transaction Commits

After a successful commit, the changes become durable. When the transaction-scoped persistence context ends, its entities are no longer managed by that context.

---

8. Why Transactions Usually Belong in the Service Layer

A repository method often represents one database action. A business operation may involve several repository calls.

```
@Transactional
public void enrollStudent(Long studentId, Long courseId) {

    Student student = studentRepository.findById(studentId);
    Course course = courseRepository.findById(courseId);

    enrollmentRepository.create(student, course);
    paymentRepository.recordPayment(student, course);
}
```

If each repository method had an independent transaction, part of the business operation could commit while a later part failed.

The service layer is normally the better transaction boundary because it represents the complete business use case.

> Repository methods perform persistence operations. Service methods define which operations must succeed or fail together.

Repository-level `@Transactional` can still be useful, but it should not accidentally divide one business operation into several unrelated transaction boundaries.

9. Which Operations Require a Transaction?

For a normal transaction-scoped JPA persistence context:

Operation	Transaction Required?	Purpose
`find()`	Not normally required without locking	Retrieve an entity by primary key
`persist()`	Yes	Make a new entity managed and schedule its insertion
`merge()`	Yes	Copy state into a managed instance
`remove()`	Yes	Mark a managed entity for deletion
`flush()`	Yes	Synchronize pending changes with the database
`refresh()`	Yes in normal application usage	Reload an entity's state from the database

A basic `find()` may work without an explicit `@Transactional`, but modifying data should be performed inside a clearly defined transaction.

10. Entity Lifecycle States

An entity instance can move through four main lifecycle states:

1. Transient
2. Managed
3. Detached
4. Removed

The managed state is central to Hibernate because dirty checking, automatic updates, first-level caching, relationship synchronization, and flushing depend on

it.

10.1 Transient State

```
Student student = new Student();
student.setName("Rahul");
student.setEmail("rahul@gmail.com");
```

This object:

- exists in Java memory,
- is not associated with a persistence context,
- is not tracked by Hibernate,
- normally does not yet represent a stored database row.

```
new Student() → TRANSIENT
```

`@Entity` makes instances of the class eligible for persistence. It does not cause every object created using `new` to be managed automatically.

10.2 Managed State

An entity is managed when it is associated with the current persistence context.

A new entity becomes managed through `persist()`:

```
entityManager.persist(student);
```

An existing entity returned by `find()` is also managed:

```
Student student =
        entityManager.find(Student.class, 1L);
```

```
Transient entity ──persist()──→ Managed entity
```

```
Database row ──find()──→ Managed entity
```

Hibernate does not create a second copy when `persist()` is called. The same Java object becomes managed.

Managed Does Not Mean "Already Written to the Database"

Calling:

```
entityManager.persist(student);
```

makes the entity managed, but the `INSERT` may be delayed until a flush. The exact timing can depend on identifier generation and other Hibernate behavior.

The term **managed** describes the object's relationship with the persistence context, not whether SQL has already executed.

10.3 Detached State

A detached entity:

- was previously managed,
- still exists as a Java object,
- may still represent an existing row,
- is no longer associated with its original persistence context.

Common ways an entity becomes detached:

The Persistence Context Ends

```
Transaction-scoped context ends
            ↓
Previously managed entities become detached
```

`detach()`

```
entityManager.detach(student);
```

This detaches one entity.

`clear()`

```
entityManager.clear();
```

This detaches every managed entity and empties the persistence context.

Changes made after detachment are not automatically detected by the original persistence context.

10.4 Removed State

A managed entity becomes removed when:

```
entityManager.remove(student);
```

Hibernate now knows that the corresponding row should be deleted when changes are flushed.

`remove()` is primarily a lifecycle transition. It does not necessarily mean that the `DELETE` statement executes at that exact line.

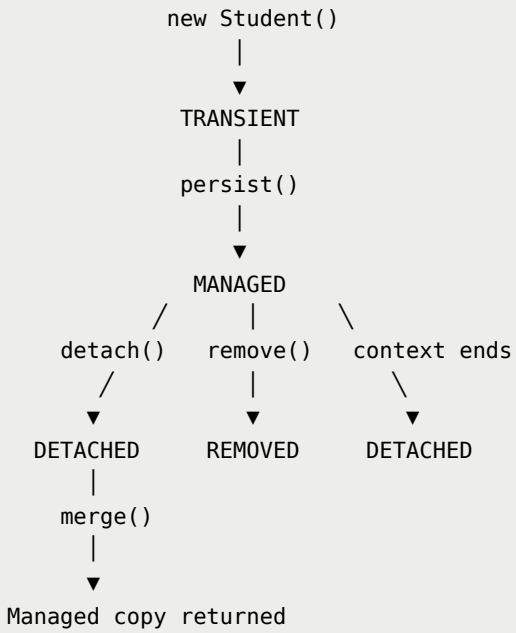
Removed vs. Detached

Detached Entity	Removed Entity
Hibernate has stopped managing it	Hibernate is managing its deletion
Its database row should remain	Its database row is scheduled for deletion
Later field changes are not dirty-checked	Deletion is synchronized during flush

`detach(student)` does not delete the student. `remove(student)` does.

---

11. Entity Lifecycle Diagram



Important Rule About `merge()`

`merge()` does not reattach the same detached Java object.

```
Student managedStudent =
        entityManager.merge(detachedStudent);
```

Hibernate copies the detached object's state into a managed instance and returns that managed instance.

```
detachedStudent  → remains detached
managedStudent   → managed return value
```

Always use the object returned by `merge()` when further managed operations are required.

---

12. Checking Whether an Entity Is Managed

Use `contains()`:

```
Student student =
        entityManager.find(Student.class, 1L);

boolean managed =
        entityManager.contains(student);
```

Inside the same persistence context, the result is normally:

```
true
```

After:

```
entityManager.detach(student);
```

the result becomes:

```
false
```

---

13. Managed Entity Identity

Within one persistence context, Hibernate maintains one managed object for a particular entity type and primary key.

```
Student first =
        entityManager.find(Student.class, 1L);

Student second =
        entityManager.find(Student.class, 1L);
```

Within the same persistence context:

```
first == second   // normally true
```

Both references point to the same managed Java object.

This is called the **identity map** principle:

> Within one persistence context, one database identity is represented by one managed entity instance.

Without this rule, two objects representing the same row could contain conflicting values inside the same unit of work.

---

14. Dirty Checking

In Hibernate, **dirty** does not mean invalid or corrupted. An entity is dirty when its current managed state differs from the state Hibernate previously knew.

```
Original state: name = "Rahul"
Current state:  name = "Rohan"
```

The Problem Dirty Checking Solves

```java
@Transactional
public void updateStudent(Long id) {

    Student student =
            entityManager.find(Student.class, id);

    student.setName("Rohan");
}
```

There is no call to:

```java
entityManager.update(student);
```

JPA does not even define an `EntityManager.update()` method. The object returned by `find()` is managed, so Hibernate can automatically detect its changed state and generate the required `UPDATE`.

```
find()
   ↓
Entity becomes managed
   ↓
Field value changes
   ↓
Flush occurs
   ↓
Dirty checking detects the change
   ↓
UPDATE is generated
```

When Dirty Checking Happens

Calling a setter does not normally execute SQL immediately.

```
Java object changes
        ↓
Entity remains managed in memory
        ↓
Flush occurs
        ↓
Hibernate detects relevant changes
        ↓
SQL executes
```

Hibernate can compare the entity's current state with its earlier snapshot during flushing. Depending on configuration and enhancement, Hibernate may also use more optimized change-tracking techniques.

Dirty Checking Applies Only to Managed Entities

```java
Student student =
        entityManager.find(Student.class, 1L);

entityManager.detach(student);
student.setName("Rohan");
```

The name change is not automatically written to the database because `student` is detached.

Setting the Same Value

```
student.setName("Rahul");
```

If the original value was already `"Rahul"`, Hibernate normally detects no actual state change and does not need to issue an update for that field change.

---

15. Flushing

Flush synchronizes the changes represented by the persistence context with the database by executing the required SQL statements.

```
Persistence Context
        |
        | flush
        ▼
INSERT / UPDATE / DELETE
        |
        ▼
Database
```

Flush Is Not Commit

This distinction is essential:

`flush()`	`commit()`
Sends pending SQL to the database	Successfully completes the transaction
Changes are still part of the current transaction	Changes become durable
Can be followed by rollback	Ends the transaction successfully

Example:

```
BEGIN
  Modify managed entity
  FLUSH       → UPDATE executes
  Error occurs
ROLLBACK      → update is undone
```

SQL execution does not automatically mean that a transaction has committed.

Automatic Flush

With the common `AUTO` flush mode, Hibernate normally flushes:

- before transaction commit,
- before certain queries whose results could be affected by pending changes.

Example:

```
@Transactional
public void updateStudent() {

    Student student =
            entityManager.find(Student.class, 1L);

    student.setName("Rohan");
}
```

Typical flow:

```
Transaction begins
        ↓
SELECT
        ↓
Student becomes managed
        ↓
setName("Rohan")
        ↓
Method returns
        ↓
Flush
        ↓
UPDATE
        ↓
Commit
```

Manual Flush

```
entityManager.flush();
```

Manual flushing is useful when SQL must be executed before the end of the method, for example:

- to detect a database constraint violation earlier,
- to synchronize pending changes before later database work,
- to control memory during batch processing when combined with `clear()`.

---

16. Transactional Write-Behind

Hibernate describes the persistence context as a transactional write-behind cache.

In a write-through approach, each Java change would immediately cause SQL:

```
Java change → SQL
Java change → SQL
Java change → SQL
```

With write-behind:

```
Java change ─┐
Java change ─┼─→ Persistence Context ─→ Flush ─→ SQL
Java change ─┘
```

Delaying synchronization gives Hibernate an opportunity to:

- avoid unnecessary work,
- order SQL statements,
- group operations,
- support JDBC batching,
- synchronize at a suitable point in the transaction.

Write-behind does not remove the need for transactions. It works inside the transaction so that the database can still commit or roll back the resulting SQL as one unit.

---

17. `flush()` and `clear()` in Batch Processing

Suppose we persist 100,000 entities in one loop:

```
for (int i = 0; i < 100_000; i++) {
    Student student = createStudent(i);
    entityManager.persist(student);
}
```

If every entity remains managed until the end, the persistence context can grow very large. That increases memory usage and the amount of entity-management work.

A common batching pattern is:

```
for (int i = 0; i < 100_000; i++) {

    Student student = createStudent(i);
    entityManager.persist(student);

    if ((i + 1) % 100 == 0) {
        entityManager.flush();
        entityManager.clear();
    }
}
```

The two operations serve different purposes:

Operation	Effect
`flush()`	Executes pending SQL; entities remain managed
`clear()`	Detaches all managed entities and empties the persistence context

Conceptually:

```
100 managed entities
        ↓
flush() → SQL is synchronized
        ↓
```

```
clear() → entities become detached
        ↓
Process the next batch
```

This controls persistence-context memory. Actual JDBC batching also depends on Hibernate's batch configuration.

---

18. First-Level Cache

The persistence context provides Hibernate's first-level cache behavior.

```
Persistence Context
┌──────────────────────────────────────┐
│ (Student, 1) → Student Object A      │
│ (Student, 2) → Student Object B      │
└──────────────────────────────────────┘
```

The cache key is conceptually made from:

```
Entity type + primary key
```

The first-level cache:

- is enabled by default,
- cannot be disabled like an optional external cache,
- belongs to one persistence context,
- is normally transaction-scoped in a Spring application,
- also guarantees managed entity identity.

First Lookup: Cache Miss

```java
Student first =
        entityManager.find(Student.class, 1L);
```

If `Student#1` is not in the persistence context:

```
Persistence Context miss
        ↓
SELECT from database
        ↓
Create Student Object A
        ↓
Store and manage Object A
```

Second Lookup: Cache Hit

```java
Student second =
        entityManager.find(Student.class, 1L);
```

If this happens in the same persistence context:

```
Persistence Context hit
        ↓
Return Student Object A
        ↓
No second SELECT required
```

Therefore:

```java
first == second   // normally true
```

Scope of the First-Level Cache

```java
@Transactional
public void methodA() {
    entityManager.find(Student.class, 1L); // SELECT
    entityManager.find(Student.class, 1L); // first-level cac
he hit
}
```

In a later independent transaction:

```
@Transactional
public void methodB() {
    entityManager.find(Student.class, 1L); // SELECT again
}
```

This is expected because the second transaction normally uses a different persistence context.

> First-level cache is not an application-wide cache. It is local to one persistence context.

Important Query Limitation

Primary-key lookups such as `find()` can be satisfied directly from the persistence context.

A JPQL or Criteria query may still execute SQL even if matching entities are already managed. Hibernate will preserve identity by returning the existing managed instances where applicable, but the query itself is not automatically skipped in every case.

---

19. How `clear()` and `detach()` Affect the Cache

`clear()`

```
Student first =
        entityManager.find(Student.class, 1L);

entityManager.clear();

Student second =
        entityManager.find(Student.class, 1L);
```

`clear()` removes all managed entities from the persistence context. The second `find()` may therefore execute another `SELECT`.

```
first == second   // normally false
```

`first` is the old detached object, while `second` is a new managed object.

`detach()`

```
Student student =
        entityManager.find(Student.class, 1L);

entityManager.detach(student);
```

Only that entity is detached. A later lookup may need to load and manage the entity again.

Method	Scope
`detach(entity)`	One entity
`clear()`	Every managed entity

20. Refreshing an Entity

The first-level cache can contain values that are older than the current database values if another transaction or external process changes the row.

Suppose the managed entity contains:

```
name = "Rahul"
```

Another process changes the database value to:

```
name = "Aman"
```

The managed Java object does not update automatically. Its value can be reloaded using:

```
entityManager.refresh(student);
```

Conceptually:

```
Current managed state
        ↓ replaced
Latest database state
        ↓
Managed entity now contains fresh values
```

`refresh()` discards unsynchronized in-memory changes for the refreshed state and replaces them with database values, so it should be used deliberately.

---

21. Quick Reference: Important `EntityManager` Methods

Method	Main Effect
`persist(entity)`	Makes a new entity managed and schedules insertion
`find(Type.class, id)`	Finds by primary key and returns a managed entity
`merge(entity)`	Copies state into a managed instance and returns it
`remove(entity)`	Marks a managed entity for deletion
`contains(entity)`	Checks whether the instance is managed
`detach(entity)`	Detaches one entity
`clear()`	Detaches all entities
`flush()`	Synchronizes pending changes with the database
`refresh(entity)`	Replaces current entity state with database state

---

22. Common Misconceptions

“An `@Entity` object is automatically managed.”

No. `new Student()` is transient until it becomes associated with a persistence context.

“Calling a setter immediately executes an `UPDATE`.”

No. A setter changes the Java object. Hibernate normally detects the change during flush.

“Managed means the SQL has already executed.”

No. Managed describes the entity’s relationship with the persistence context.

“`flush()` and `commit()` are the same.”

No. Flush executes pending SQL inside the current transaction. Commit completes the transaction.

“`detach()` deletes the database row.”

No. It only stops the current persistence context from managing that object.

“`merge()` makes the same detached object managed.”

No. It returns a managed instance containing copied state. The supplied object remains detached.

“The first-level cache is shared across the application.”

No. It belongs to a single persistence context.

“A singleton repository shares one physical `EntityManager` across all requests.”

Normally no. Spring injects a proxy that routes calls to the transaction-associated `EntityManager`.

---

23. Complete Mental Model

```
@Transactional service method begins
                ↓
Spring starts or joins a transaction
                ↓
EntityManager is associated with the transaction
                ↓
Persistence Context manages loaded/new entities
                ↓
```

```
Business code changes managed objects
                ↓
Dirty checking detects state changes
                ↓
Flush converts changes into SQL
                ↓
Database transaction commits or rolls back
                ↓
Transaction-scoped Persistence Context ends
                ↓
Previously managed objects become detached
```

The central idea is:

> Hibernate does not simply translate individual method calls into SQL. It manages entity state inside a persistence context and synchronizes that state with the database within a transaction.